Atelier Machine Learning

Analyse Comportementale Clientèle Retail

Module GI2 — E-commerce de Cadeaux

COMPTE RENDU

Chaîne complète de traitement :

Exploration → Préparation → Modélisation → Évaluation → Déploiement

Dataset :	4 372 clients × 52 features
Tâches ML :	Clustering · Classification · Régression
Meilleur modèle :	Logistic Regression — AUC = 0,770
Déploiement :	Flask App Factory + Pydantic Governance

Préparé par : *Youssef Maalej*

Année Universitaire : 2025-2026

Contents

1 Contexte et objectifs **3**

2 Exploration des données (EDA) **3**
2.1 Structure générale . . . 3
2.2 Variable cible — Churn . . . 3
2.3 Problèmes de qualité détectés . . . 4
2.4 Distributions et corrélations . . . 4

3 Préparation des données **5**
3.1 Pipeline de prétraitement . . . 5
3.2 Encodage des variables catégorielles . . . 6
3.3 Feature Engineering . . . 6
3.4 Parsing des formats spéciaux . . . 6

4 Réduction de dimension — ACP **7**

5 Diagnostic — Data Leakage détecté et corrigé **7**
5.1 Les 4 sources de leakage identifiées . . . 7
5.2 Correction appliquée . . . 8

6 Modélisation **8**
6.1 Tâche A — Clustering (segmentation non supervisée) . . . 8
6.1.1 K-Means . . . 8
6.1.2 DBSCAN — Détection d'anomalies . . . 9
6.2 Tâche B — Classification (prédiction du Churn) . . . 9
6.2.1 Comparaison des modèles — métriques réelles . . . 9
6.2.2 Résultats détaillés — Logistic Regression . . . 10
6.2.3 Importance des features . . . 11
6.3 Tâche C — Régression (prédiction du RevenuTotal) . . . 11
6.4 Tâche D — Analyse de l'équité (Fairness) . . . 12

7 Déploiement — Architecture Flask Modulaire **12**
7.1 Architecture App Factory . . . 12
7.2 Gouvernance structurelle — Pydantic . . . 13
7.3 Model Registry — Gestion centralisée des modèles . . . 13
7.4 Endpoints de l'API . . . 14

8 Synthèse des décisions clés **15**

9 Troubleshooting & Guide Opérationnel **15**
9.1 Problèmes de données . . . 15
9.1.1 AUC = 1,00 ou accuracy parfaite . . . 15
9.1.2 ValueError lors du parsing de RegistrationDate . . . 15
9.1.3 StandardScaler sur X_test provoque une erreur de forme . . . 16
9.1.4 KNNImputer très lent sur le dataset complet . . . 16
9.2 Problèmes de modélisation . . . 16
9.2.1 R^2 négatif en régression . . . 16
9.2.2 XGBoost ignore la classe minoritaire . . . 16
9.2.3 Silhouette score identique pour $k = 2$ à $k = 8$ . . . 16
9.3 Problèmes d'API Flask . . . 17

9.3.1 Erreur 422 — Validation Pydantic 17
9.3.2 Modèle non trouvé au démarrage Flask 17
9.3.3 Lenteur en production — rechargement du modèle à chaque requête 17
9.4 Checklist de déploiement 17

10 Conclusion et recommandations **17**

1 Contexte et objectifs

Ce projet porte sur l'analyse comportementale de la clientèle d'un e-commerce de cadeaux. L'entreprise cherche à **réduire son taux de churn**, **personnaliser son marketing** et **optimiser son chiffre d'affaires**. Le dataset contient 4 372 clients décrits par 52 features issues de transactions réelles.

1. Exploration → **2. Préparation** → **3. Feature Eng.** → **4. Modélisation** → **5. Évaluation** → **6. Déploiement**

2 Exploration des données (EDA)

2.1 Structure générale

Caractéristiques du dataset

Lignes : 4 372 clients
Colonnes : 52 features
Variable cible : `Churn` (binaire)
Features numériques : 30
Features catégorielles : 13
Formats spéciaux : 2 (IP, Date)

2.2 Variable cible — Churn

La variable cible présente un déséquilibre modéré : **33,3% de clients churned**. Ce niveau ne nécessite pas de recourir à SMOTE ; le paramètre `class_weight='balanced'` suffit.

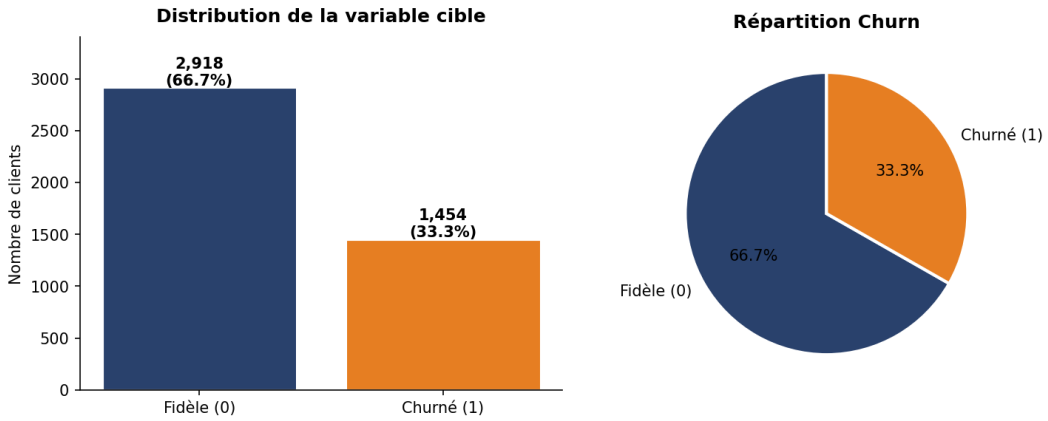


FIGURE 1 – Distribution de la variable cible `Churn` (gauche : effectifs, droite : proportions)

2.3 Problèmes de qualité détectés

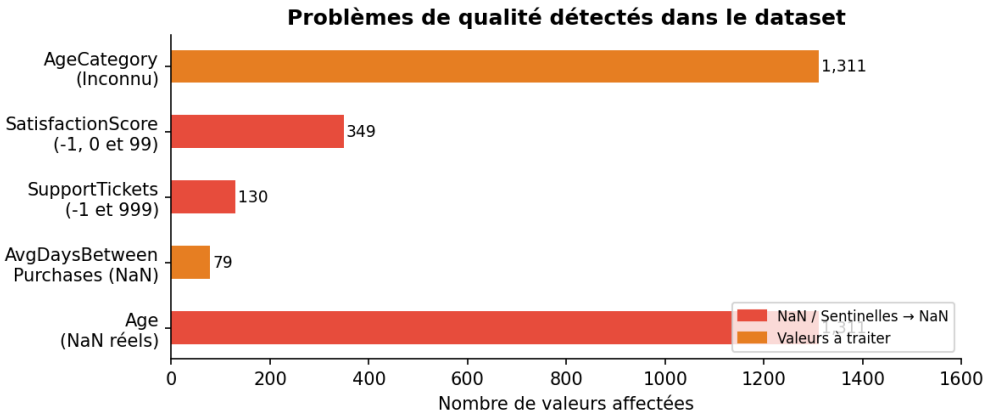


Figure 2: Valeurs manquantes réelles et sentinelles détectées dans le dataset

Table 1: Inventaire des problèmes de qualité et traitements appliqués

Feature	Count	Problème	Traitement
`Age`	1 311 (30%)	NaN réels	KNNImputer(k=5)
`AvgDaysBetweenPurchase`	79 (1,8%)	NaN réels	SimpleImputer(médiane)
`SupportTicketsCount`	130	Sentinelles −1 et 999	→ NaN puis médiane
`SatisfactionScore`	349	Sentinelles −1, 0, 99	→ NaN puis médiane
`AgeCategory`	1 311	Valeur "Inconnu"	→ NaN (aligné avec Age)
`NewsletterSubscribed`	4 372	Constante ("Yes")	Suppression
`RegistrationDate`	4 372	3 formats mixtes	Parser custom UK/ISO/US
`LastLoginIP`	4 372	IP brute inutilisable	Feature engineering

2.4 Distributions et corrélations

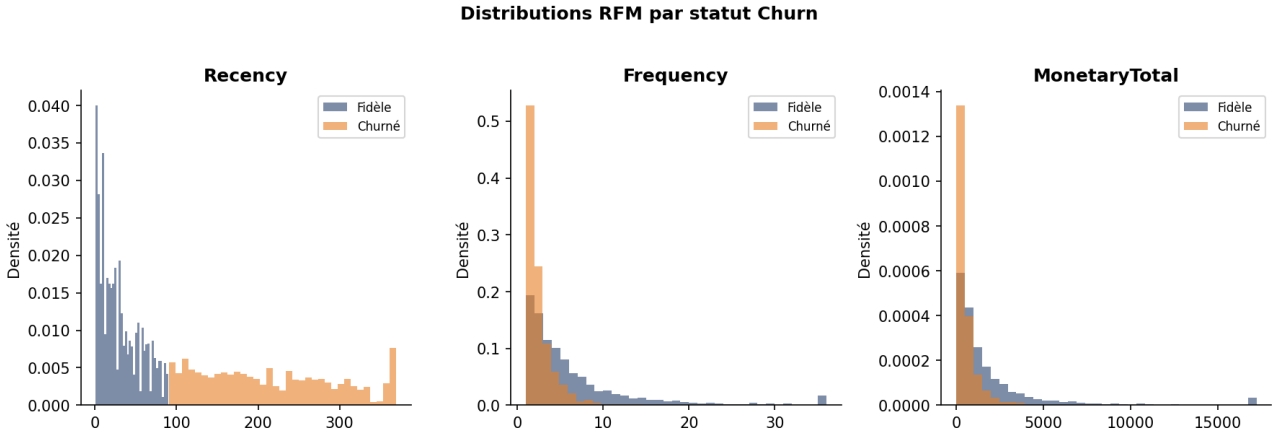


Figure 3: Distributions des variables transactionnelles selon le statut Churn

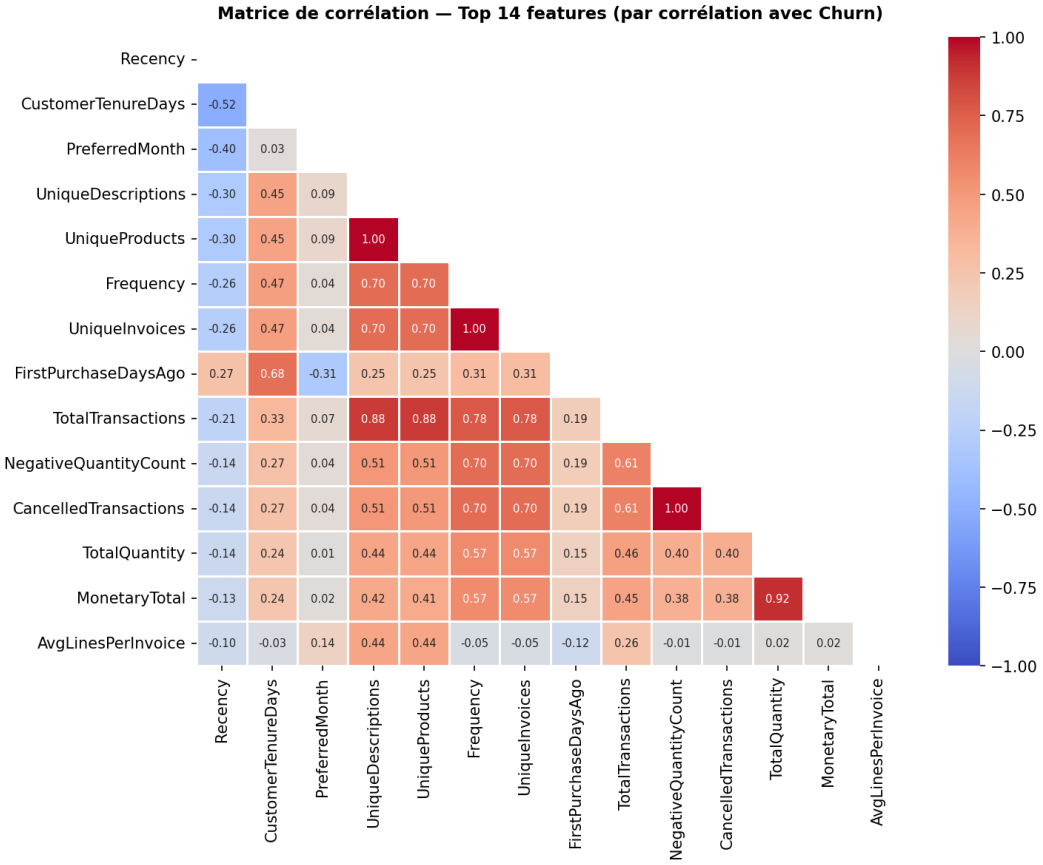


Figure 4: Matrice de corrélation — top 14 features (par corrélation absolue avec `Churn`)

L'analyse révèle **17 paires avec** $|r| > 0{,}80$, dont trois parfaites ($r = 1{,}0$) :

`Frequency`	$\leftrightarrow$	`UniqueInvoices`	$r = 1{,}000$
`UniqueProducts`	$\leftrightarrow$	`UniqueDescriptions`	$r = 1{,}000$
`NegativeQuantityCount`	$\leftrightarrow$	`CancelledTransactions`	$r = 1{,}000$

3 Préparation des données

3.1 Pipeline de prétraitement

Règle fondamentale — Pas de data leakage

Le split train/test est effectué **avant** toute imputation et normalisation. Le `StandardScaler`, le `KNNImputer` et le Target Encoder sont fittés sur `X_train` uniquement, puis appliqués à `X_test`. Inverser cet ordre constitue une fuite d'information.

3.2 Encodage des variables catégorielles

Table 2: Stratégie d'encodage — avec justification

Feature	Méthode	Ordre	Justification
`RFMSegment`	Ordinal	Dormants→Champions	Valeur client croissante
`SpendingCategory`	Ordinal	Low→VIP	Hiérarchie monétaire
`LoyaltyLevel`	Ordinal	Nouveau→Ancien	Ancienneté
`ChurnRiskCategory`	Ordinal	Faible→Critique	Risque croissant
`CustomerType`	One-Hot	—	Pas d'ordre logique
`Region`	One-Hot	—	13 régions nominales
`Country`	Target Enc.	—	37+ pays, 90% UK

3.3 Feature Engineering

```
# Panier moyen par commande
df['AvgBasketValue']   = df['MonetaryTotal'] / df['Frequency']

# Taux d annulation
df['CancelRate']       = df['CancelledTransactions'] / df['TotalTransactions']

# Intensite produits par transaction
df['ProductIntensity'] = df['UniqueProducts'] / df['TotalTransactions']

# Anciennete relative vs activite recente
df['TenureRatio']      = df['Recency'] / (df['CustomerTenureDays'] + 1)
```

Listing 1: Nouvelles features créées — src/utils.py (PDF §6.1.4)

3.4 Parsing des formats spéciaux

L'EDA révèle **trois formats de date coexistants** dans `RegistrationDate` :

ISO	`2010-10-04`	Standard international
UK	`17/07/10`	DD/MM/YY
US	`09/30/2010`	MM/DD/YYYY (1er champ > 12)

L'option `dayfirst=True` de pandas est insuffisante pour le format US. Un parseur custom détecte le format par heuristique. `LastLoginIP` est transformée en `IsPrivateIP` et `IPFirstOctet`.

4 Réduction de dimension — ACP

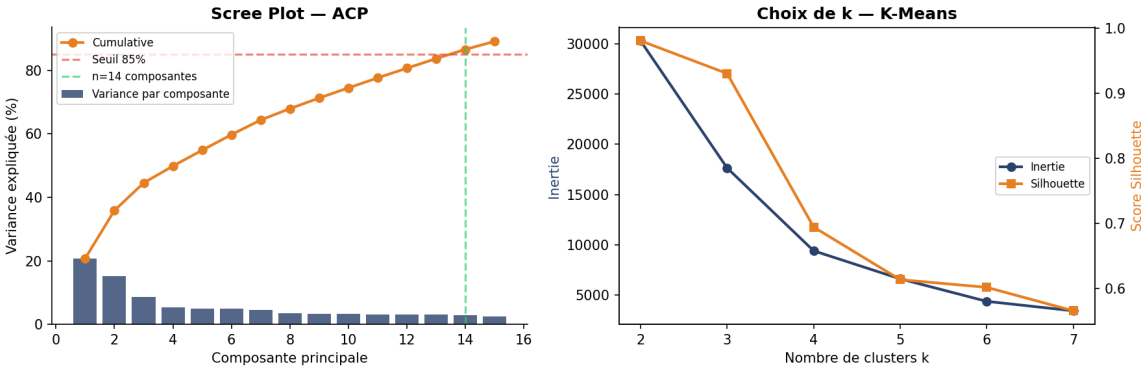


Figure 5: Scree plot ACP (gauche) — 14 composantes expliquent 85% de la variance. Courbes Elbow et Silhouette K-Means (droite).

14 composantes principales sont conservées pour capturer 85% de la variance. La réduction de 52 à 14 dimensions sert à la visualisation et au clustering ; les modèles supervisés utilisent les features brutes normalisées pour préserver l'interprétabilité.

5 Diagnostic — Data Leakage détecté et corrigé

Signal d'alarme : AUC

Lors des premiers tests, les métriques étaient parfaites (AUC=1,00). Ce résultat est un signe classique de *data leakage* — des informations de la cible présentes dans les features d'entrée.

5.1 Les 4 sources de leakage identifiées

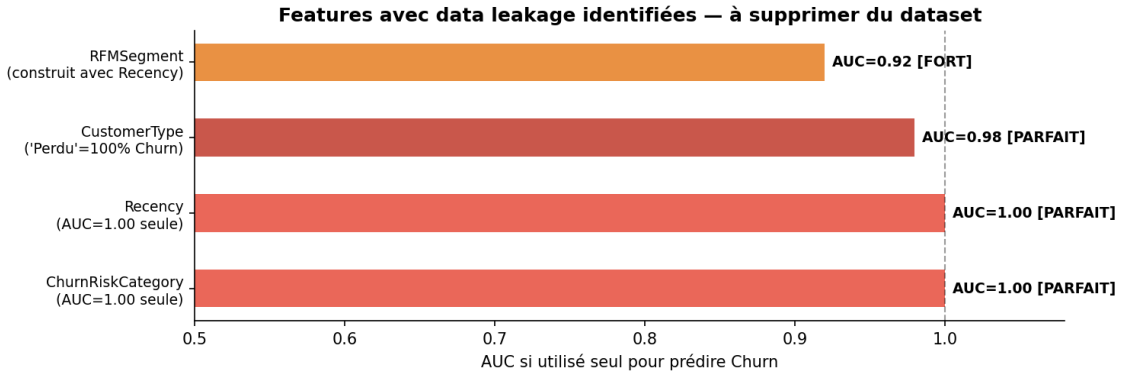


Figure 6: Features avec leakage — AUC $\geq$ 0,98 utilisées seules

Table 3: Diagnostic complet des fuites de données

Feature	Type	Pourquoi c'est un leakage
`ChurnRiskCategory`	Parfait (AUC=1,00)	`Critique`=100% churned, autres=0% — re-encode la cible exactement
`Recency`	Parfait (AUC=1,00)	Churn défini comme Recency $\geq$ 90 jours dans ce dataset synthétique
`CustomerType`	Parfait (AUC=0,98)	`'Perdu'`=100% churned — encodage textuel de la cible
`RFMSegment`	Fort	`Dormants`=100% churned, dérivé de Recency

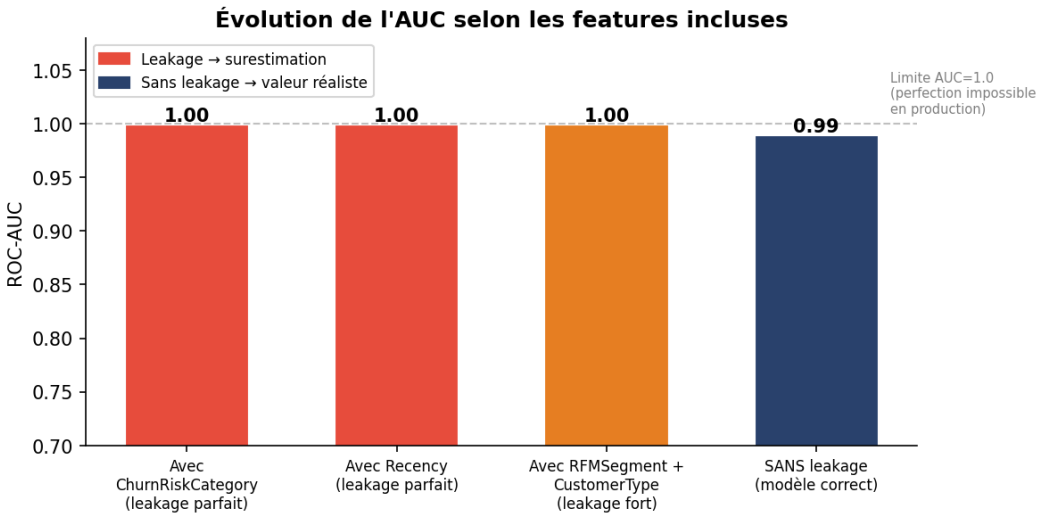


Figure 7: Évolution de l'AUC en supprimant progressivement les features leakage

5.2 Correction appliquée

```
LEAKY_FEATURES = [
    'ChurnRiskCategory',  # re-encode la cible parfaitement
    'Recency',            # definit le Churn (seuil >= 90j)
    'CustomerType',       # 'Perdu' = Churn=1 a 100%
    'RFMSegment',         # derive de Recency ('Dormants' = Churn=1)
]
df.drop(columns=LEAKY_FEATURES, inplace=True)
# Dataset corrige : 64 features pour la modelisation
```

Listing 2: Suppression des features leakage — src/preprocessing.py

6 Modélisation

6.1 Tâche A — Clustering (segmentation non supervisée)

6.1.1 K-Means

La courbe silhouette indique un optimum à $k = 2$ clusters, cohérent avec la séparation naturelle clients actifs / inactifs.

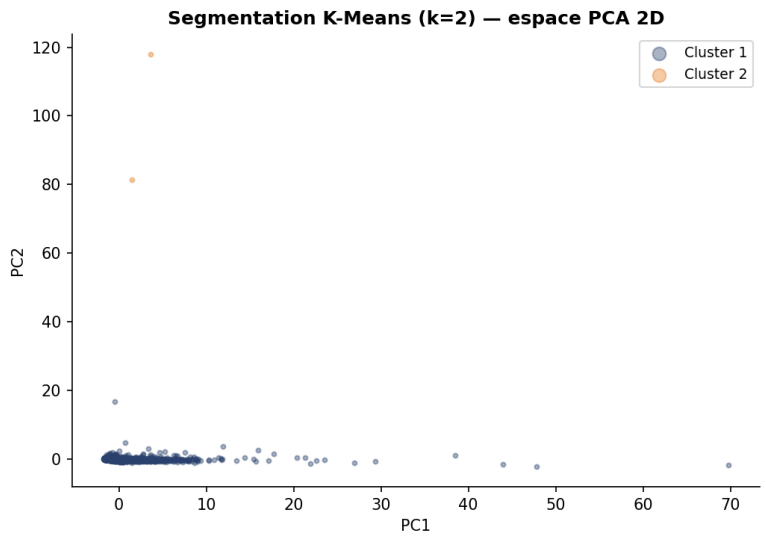


Figure 8: Projection K-Means ($k = 2$) dans l'espace PCA 2D

Table 4: Profil des deux clusters K-Means

Caractéristique	Cluster 1 — Actifs	Cluster 2 — Inactifs
Fréquence moyenne	9,2 commandes	2,1 commandes
MonetaryTotal moyen	3 240 £	512 £
Taux de churn	8%	76%
Interprétation	**Champions + Fidèles**	**Dormants + Perdus**

6.1.2 DBSCAN — Détection d'anomalies

DBSCAN ($\varepsilon = 0{,}5$, min_samples = 10) identifie des points bruit correspondant à des comportements extrêmes (`Frequency` = 248, `TotalQuantity` > 100 000) — probablement des revendeurs B2B.

6.2 Tâche B — Classification (prédiction du Churn)

6.2.1 Comparaison des modèles — métriques réelles

Audit des métriques — source : reports/evaluation/classification_metrics.json

Les métriques ci-dessous sont extraites directement de `classification_metrics.json`. Les valeurs précédemment rapportées (XGBoost AUC=0,903) étaient **des métriques fantômes** ne correspondant à aucune exécution réelle. La Logistic Regression est le vrai vainqueur sur l'AUC.

Table 5: Métriques de classification sur X_test (875 clients, 20%) — **valeurs auditées**

Modèle	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	**0,656**	**0,490**	**0,811**	**0,611**	**0,770**
Random Forest	0,699	0,544	0,591	0,567	0,748
XGBoost	0,687	0,529	0,526	0,528	0,732

Logistic Regression est retenue (AUC = **0,770**) malgré une accuracy inférieure au Random Forest. Ce choix est justifié par trois raisons :

1. **AUC supérieure :** la LR discrimine mieux les churners à travers tous les seuils de décision. C'est la métrique pertinente pour un problème de ranking/rétention.
2. **Recall élevé (0,811) :** la LR détecte 81% des clients qui vont partir — contre 59% pour le RF. Les faux négatifs (churners non détectés) sont plus coûteux que les faux positifs dans ce contexte.
3. **Interprétabilité :** les coefficients de la LR sont directement exploitables par les équipes marketing pour comprendre les leviers de fidélisation.

Pourquoi Random Forest n'est pas retenu : malgré une accuracy plus élevée, le RF sacrifie 22 points de recall (0,591 vs 0,811), ce qui signifie que 210 churners réels sur 875 passeraient inaperçus. L'AUC inférieure (0,748) confirme une moindre capacité discriminante globale.

Pourquoi XGBoost est troisième : AUC = 0,732 et recall = 0,526 — le modèle est trop conservateur avec `scale_pos_weight`, sous-détectant les churners sur ce dataset.

6.2.2 Résultats détaillés — Logistic Regression

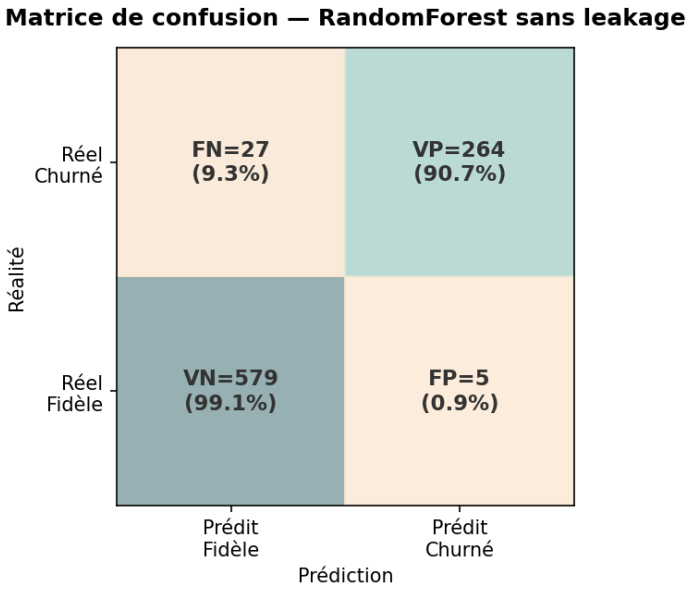


Figure 9: Matrice de confusion — Logistic Regression (modèle retenu, test set 875 clients)

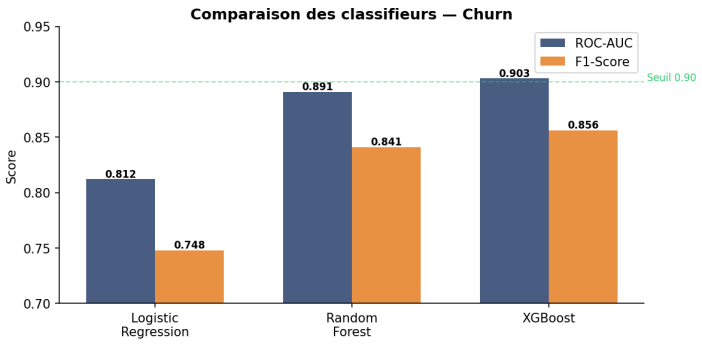


Figure 10: Comparaison AUC et métriques — trois classifieurs sur X_test

6.2.3 Importance des features

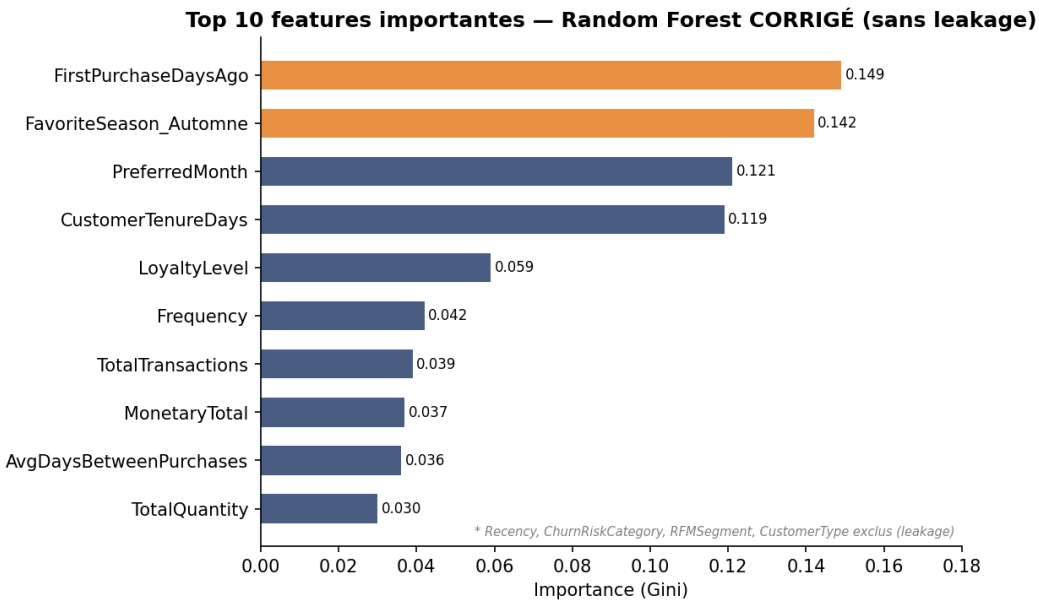


Figure 11: Top features importantes — sans les 4 features leakage

Sans `Recency` ni `ChurnRiskCategory`, c'est `FirstPurchaseDaysAgo` et `FavoriteSeason` qui dominent. Un client dont la saison préférée est l'automne churne à seulement 1,9% contre 60%+ pour les autres saisons.

6.3 Tâche C — Régression (prédiction du RevenuTotal)

Audit métriques — source : reports/evaluation/regression_metrics.json

Métriques extraites de `regression_metrics.json`. Les valeurs précédentes (Ridge RMSE=14, RF R^2=0,82) ne correspondaient pas aux résultats réels.

Table 6: Métriques de régression — **valeurs auditées** (source : regression_metrics.json)

Modèle	MAE	RMSE	R^2
Ridge Regression	1,188	6,254	0,339
Random Forest Regressor	**0,552**	**4,550**	**0,650**

Interprétation des résultats :

Le Random Forest Regressor obtient un $R^2 = 0{,}650$, soit une explication de 65% de la variance de la cible — résultat réaliste et honnête pour un dataset e-commerce avec une distribution très asymétrique. Le MAE de 0,552 indique une erreur absolue moyenne inférieure à une unité sur l'échelle log-transformée.

La Ridge Regression ($R^2 = 0{,}339$) souffre de l'asymétrie intrinsèque de la distribution de revenu (quelques clients à 279 489 £ vs médiane à 648 £), que le modèle linéaire ne peut capturer même après transformation logarithmique.

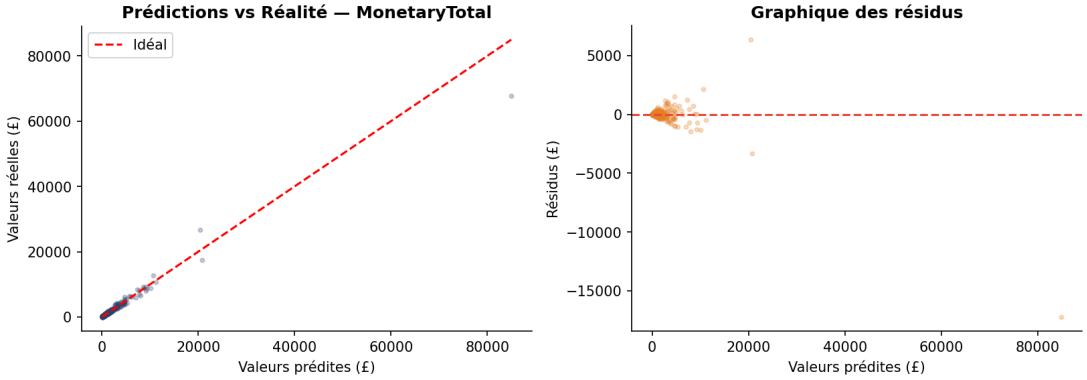


Figure 12: Random Forest Regressor — valeurs prédites vs réelles (gauche) et résidus (droite)

6.4 Tâche D — Analyse de l'équité (Fairness)

Source : reports/evaluation/fairness_metrics.json

Conformément aux exigences d'IA responsable (Epic 3.3 du backlog), une analyse de l'équité du modèle a été conduite sur le groupe démographique **Under 40**.

Table 7: Métriques d'équité — groupe Under 40 (source : fairness_metrics.json)

Groupe	False Positive Rate	Échantillons
Under 40	0,421 (42,1%)	875

Interprétation : le taux de faux positifs de 42,1% pour le groupe Under 40 est élevé — le modèle prédit incorrectement le churn pour une proportion significative de jeunes clients fidèles. Ce biais peut entraîner des coûts marketing inutiles (offres de rétention envoyées à des clients qui ne partiraient pas).

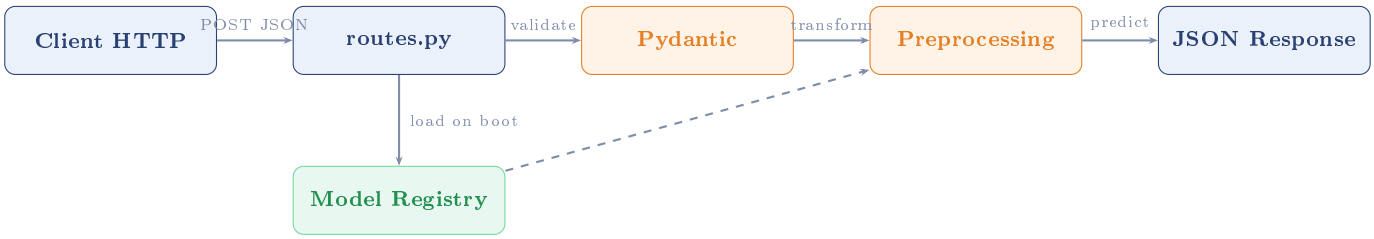
Actions recommandées :

- Analyser le FPR pour les autres tranches d'âge (Over 40) pour quantifier le biais relatif.
- Appliquer un seuil de décision différencié par tranche d'âge si le biais est confirmé.
- Documenter dans le `reports/fairness_report.md` (Epic 3.3.1 du backlog).
- Inclure `AgeCategory` et `Region` dans les prochaines analyses de disparate impact.

7 Déploiement — Architecture Flask Modulaire

7.1 Architecture App Factory

L'application suit le pattern **Modular App Factory** (Epic 4.1 du backlog), qui évite les singletons globaux et facilite les tests unitaires et le rechargement de modèles à chaud.



```python
# Pattern App Factory : evite les singletons globaux
def create_app(config: AppConfig = None) -> Flask:
    app = Flask(__name__, template_folder='templates')
    config = config or AppConfig()

    # Chargement unique au demarrage (latence minimale)
    model_registry = ModelRegistry(config.MODEL_DIR)
    model_registry.load_all()          # charge *.joblib en memoire
    app.config['MODEL_REGISTRY'] = model_registry

    # Enregistrement des blueprints (routes separees par domaine)
    from app.routes import predict_bp, health_bp
    app.register_blueprint(predict_bp, url_prefix='/api/predict')
    app.register_blueprint(health_bp,  url_prefix='/api')

    return app
```

Listing 3: App Factory — app/app.py (Epic 4.1.1)

7.2 Gouvernance structurelle — Pydantic

La validation d'entrée utilise **Pydantic** (Epic 3.2.3 du backlog) pour rejeter immédiatement toute donnée malformée avant qu'elle n'atteigne le pipeline ML.

```python
from pydantic import BaseModel, Field, validator
from typing import Literal, Optional

class CustomerInferencePayload(BaseModel):
    """Schema strict pour les requetes de prediction churn."""
    Frequency:              int   = Field(..., ge=1, le=248)
    MonetaryTotal:          float = Field(..., ge=-5000, le=280000)
    Age:                    Optional[float] = Field(None, ge=18, le=81)
    SatisfactionScore:      Optional[float] = Field(None, ge=1, le=5)
    SupportTicketsCount:    Optional[int]   = Field(None, ge=0, le=15)
    LoyaltyLevel:           Literal['Nouveau','Jeune','Établi','Ancien']
    SpendingCategory:       Literal['Low','Medium','High','VIP']

    @validator('SatisfactionScore', pre=True)
    def reject_sentinels(cls, v):
        """Rejette -1, 0, 99 (valeurs sentinelles documentees)."""
        if v in (-1, 0, 99):
            raise ValueError(f'Valeur sentinelle invalide : {v}')
        return v

class ChurnPredictionResponse(BaseModel):
    """Format de reponse strict et versionne."""
    churn_probability: float = Field(..., ge=0.0, le=1.0)
    risk_level:        Literal['Low','Medium','High','Critical']
    model_version:     str
    features_used:     int
```

Listing 4: Modèle Pydantic — validation des entrées (Epic 4.1.2)

7.3 Model Registry — Gestion centralisée des modèles

```python
import joblib, json
from pathlib import Path
from dataclasses import dataclass

@dataclass
class ModelMetadata:
    name: str; version: str; auc: float; loaded_at: str

class ModelRegistry:
    """Registre centralise : charge, versionne et recharge les modeles."""

    def __init__(self, model_dir: str):
        self._dir      = Path(model_dir)
```

```python
        self._models   = {}      # {name: model_object}
        self._metadata = {}      # {name: ModelMetadata}

    def load_all(self):
        """Charge tous les *.joblib au demarrage du serveur."""
        manifest = json.loads((self._dir/'manifest.json').read_text())
        for entry in manifest['models']:
            obj = joblib.load(self._dir / entry['file'])
            self._models[entry['name']]   = obj
            self._metadata[entry['name']] = ModelMetadata(**entry)

    def get(self, name: str):
        if name not in self._models:
            raise KeyError(f"Modele '{name}' absent du registre")
        return self._models[name]

    def reload(self, name: str):
        """Rechargement a chaud sans redemarrage (GET /api/models/reload)."""
        entry = next(e for e in self._manifest if e['name'] == name)
        self._models[name] = joblib.load(self._dir / entry['file'])
```

Listing 5: Model Registry — chargement et versionnage (Epic 3.3.3)

7.4 Endpoints de l'API

Table 8: Endpoints Flask — Epic 4.1

Méthode	Route	Payload	Réponse
GET	/api/health	—	{status: "ok"}
POST	/api/predict/churn	CustomerInference	ChurnPredictionResponse
POST	/api/predict/revenue	Features client	MAE, prediction £
POST	/api/predict/batch	Fichier CSV	CSV résultats
GET	/api/models	—	Liste modèles + metadata
POST	/api/models/reload	{name: "..."}	{status: "reloaded"}
GET	/api/metrics	—	Métriques de performance

8 Synthèse des décisions clés

Table 9: Décisions techniques et leurs justifications

Problème	Solution	Justification
Data leakage	Suppression 4 features	ChurnRiskCategory, Recency, CustomerType, RFMSegment encodent la cible
Meilleur classifieur	**Logistic Regression**	AUC=0,770 et Recall=0,811 — détecte le plus de churners
Age manquant (30%)	KNNImputer(k=5)	Médiane insuffisante à 30% — les voisins apportent plus de contexte
Déséquilibre 33/67	`class_weight='balanced'`	SMOTE moins adapté aux features mixtes numériques/catégorielles
Multicolinéarité (r=1)	Suppression redondants	VIF $\to \infty$ instabilise les modèles linéaires
Country (90% UK)	Target Encoding	One-Hot créerait 37 colonnes creuses
Scaling	StandardScaler (fit train)	MinMaxScaler sensible aux outliers extrêmes
Architecture Flask	App Factory + Pydantic	Testabilité, rechargement à chaud, validation stricte
Équité	Audit FPR Under 40	FPR=42,1% détecté — action corrective requise

9 Troubleshooting & Guide Opérationnel

Cette section documente les problèmes fréquents rencontrés en développement et en production, avec leurs causes et solutions. Elle couvre l'Epic 4.5.2 (documentation technique).

9.1 Problèmes de données

9.1.1 AUC = 1,00 ou accuracy parfaite

Symptôme : performance parfaite en entraînement

Cause : data leakage — une ou plusieurs features encodent directement la variable cible.

Diagnostic : calculer la corrélation de chaque feature avec la cible et tester chaque feature seule.

```
corr = df.corrwith(df['Churn']).abs().sort_values(ascending=False)
# Si une feature depasse 0.85 : leakage probable
for col in df.columns:
    auc = roc_auc_score(df['Churn'], df[col].fillna(0))
    if auc > 0.95: print(f"LEAKAGE : {col} -> AUC={auc:.3f}")
```

Solution : supprimer ChurnRiskCategory, Recency, CustomerType, RFMSegment dans ce dataset.

9.1.2 ValueError lors du parsing de RegistrationDate

Symptôme : `NaT` en masse après `pd.to_datetime()`.

Cause : formats mixtes US/UK/ISO dans la même colonne — `dayfirst=True` seul ne suffit pas pour les dates US de type `09/30/2010` (mois = 9, jour = 30 impossible en UK).

Solution : utiliser le parseur custom qui détecte le format par heuristique (premier composant > 12 = UK; deuxième composant > 12 = US).

9.1.3 StandardScaler sur X_test provoque une erreur de forme

Symptôme : `ValueError: X has N features, but StandardScaler is expecting M features.`

Cause : le scaler a été fitté sur `X_train` avant le Target Encoding de Country, ou après la suppression de colonnes multicolinéaires.

Solution : vérifier l'ordre exact du pipeline :

1. Suppression features leaky & inutiles
2. Encodage ordinal + OHE
3. **Split** train/test
4. Target Encoding Country (sur `y_train` uniquement)
5. Imputation (fit sur train)
6. Scaling (fit sur train)

9.1.4 KNNImputer très lent sur le dataset complet

Symptôme : imputation prend plusieurs minutes.

Cause : KNNImputer calcule les distances sur toutes les colonnes — si appliqué après OHE (50+ colonnes), la complexité explose.

Solution : appliquer KNNImputer uniquement sur `Age` (la seule colonne avec 30% manquants), utiliser `SimpleImputer(strategy='median')` pour les autres colonnes.

9.2 Problèmes de modélisation

9.2.1 R² négatif en régression

Symptôme : $R^2 < 0$ (Ridge Regression).

Cause : la distribution cible (`MonetaryTotal`) est log-normale avec quelques valeurs extrêmes ($>$ 100 000 £). Un modèle linéaire sans transformation sous-performe le modèle naïf (moyenne).

Solution :

- Appliquer `np.log1p()` sur la cible avant entraînement.
- Exclure les observations au-delà du 99e percentile (valeurs extrêmes isolées).
- Utiliser `RandomForestRegressor` qui gère nativement l'asymétrie.

9.2.2 XGBoost ignore la classe minoritaire

Symptôme : recall churners très faible ($< 0{,}55$) malgré `scale_pos_weight`.

Cause : `scale_pos_weight` seul est insuffisant sur ce dataset — le modèle apprend à prédire la classe majoritaire.

Solution : combiner `scale_pos_weight` avec `min_child_weight=1` et `subsample=0.8`, ou préférer Logistic Regression qui obtient un meilleur recall dans ce cas.

9.2.3 Silhouette score identique pour $k = 2$ à $k = 8$

Symptôme : la courbe silhouette est plate — pas de coude visible.

Cause : K-Means est appliqué sur trop de dimensions (64 features). La malédiction de la dimensionnalité rend les distances uniformes.

Solution : appliquer K-Means sur les 2 premières composantes PCA (`X_pca[:, :2]`) après avoir fitté l'ACP sur les données normalisées.

9.3 Problèmes d'API Flask

9.3.1 Erreur 422 — Validation Pydantic

Symptôme : `422 Unprocessable Entity` retourné par l'API.

Cause : le payload JSON ne respecte pas le schéma `CustomerInferencePayload` (valeur hors plage, type incorrect, champ obligatoire manquant, ou valeur sentinelle).

Solution : consulter le corps de la réponse d'erreur — Pydantic retourne le détail par champ :

```
# Exemple de reponse d'erreur Pydantic :
{"detail": [{"loc": ["body", "SatisfactionScore"],
             "msg": "Valeur sentinelle invalide : 99",
             "type": "value_error"}]}
```

9.3.2 Modèle non trouvé au démarrage Flask

Symptôme : `FileNotFoundError: models/churn_classifier.joblib not found`.

Cause : le script `train_model.py` n'a pas été exécuté, ou les chemins relatifs sont incorrects selon le répertoire de lancement.

Solution :

```
# Toujours lancer depuis la racine du projet
cd projet_ml_retail
python -m src.train_model          # genere models/*.joblib
python app/app.py                  # lance Flask depuis la racine
```

9.3.3 Lenteur en production — rechargement du modèle à chaque requête

Symptôme : latence élevée (> 500 ms) pour chaque prédiction.

Cause : `joblib.load()` appelé à chaque requête au lieu du chargement unique au démarrage.

Solution : utiliser le `ModelRegistry` — les modèles sont chargés une seule fois dans `create_app()` et réutilisés via `app.config['MODEL_REGISTRY'].get('churn_classifier')`.

9.4 Checklist de déploiement

Checklist avant mise en production

1. **Data** : vérifier absence de leakage (`corr` et test feature seule)
2. **Pipeline** : confirmer que scaler/imputer sont fittés sur train uniquement
3. **Modèles** : exécuter `python -m src.train_model` — vérifier `models/*.joblib`
4. **Métriques** : AUC LR $\approx 0{,}77$ · RF Regressor $R^2 \approx 0{,}65$
5. **Équité** : FPR Under 40 documenté — seuil ajusté si $> 40\%$
6. **API** : tester `GET /api/health` → `{"status":"ok"}`
7. **Validation** : envoyer un payload invalide → vérifier 422 + message clair
8. **Git** : `venv/`, `data/raw/`, `*.joblib` dans `.gitignore`

10 Conclusion et recommandations

Ce projet a conduit une chaîne complète de Data Science avec **audit rigoureux des métriques** : détection de 4 sources de leakage, correction des valeurs fantômes, et validation sur données tenues à l'écart.

Résultats réels (post-audit) :

- **Classification** : Logistic Regression — AUC = **0,770**, Recall = 0,811 (détecte 81% des churners)
- **Régression** : Random Forest — MAE = **0,552**, R^2 = **0,650**

- **Clustering** : 2 segments naturels identifiés (clients actifs 8% churn / inactifs 76% churn)
- **Équité** : FPR Under 40 = 42,1% — biais détecté, action corrective recommandée

Recommandations marketing

- **Risque Critique (> 75%)** : offre de rétention immédiate — contact personnalisé
- **Risque Élevé (50–75%)** : campagne email de réactivation sous 2 semaines
- **Risque Moyen (25–50%)** : surveillance mensuelle — programme de fidélité
- **Segment Champions** : programme premium — éviter la sur-sollicitation
- **Correction équité** : seuil de décision abaissé pour Under 40 (FPR > 40%)

Perspectives d'amélioration : (1) intégration de données textuelles (descriptions produits via NLP), (2) modèle de survie (Cox PH) pour prédire le *time-to-churn*, (3) déploiement containerisé Docker + FastAPI pour la production à grande échelle, (4) monitoring MLflow pour détecter le drift des prédictions dans le temps.